

PI MODULES NEXLANCE

MODULE 1 : JOB OFFER (Gestion des Offres)

ROLE : CLIENT

Créer une Offre

`/jobs/create`

Objectif

Créer une nouvelle offre.

Inputs

- Titre
- Description (rich text)
- Catégorie
- Budget
- Type budget (fixed / hourly)
- Durée estimée
- Deadline
- Compétences requises
- Niveau expérience
- Localisation
- Remote (checkbox)
- Pièces jointes

Actions

- Sauvegarder brouillon

- Publier
 - Annuler
-

Mes Offres

`/my-jobs`

Affichage (Cartes ou Tableau)

- Titre
- Catégorie
- Budget
- Statut
- Nombre candidatures
- Nombre vues
- Date publication

Actions

- Voir
 - Modifier
 - Archiver
-

Détail Offre + Gestion Candidatures

`/my-jobs/:id`

Section Info Offre

- Description complète
- Budget
- Skills
- Deadline
- Statut

Onglets Candidatures

- Toutes
- Pending
- Shortlisted
- Accepted
- Rejected

Carte Candidature

- Photo freelance
- Nom
- Rating
- Tarif proposé
- Date disponibilité
- Extrait lettre

Actions

- Voir profil
- Shortlist
- Accepter
- Rejeter

ROLE : FREELANCE

Rechercher Offres

</browse-jobs>

Filtres

- Catégorie
- Budget
- Type
- Expérience

- Remote
- Date publication

Carte Offre

- Titre
 - Budget
 - Skills
 - Nombre candidatures
 - Date
 - Bouton Postuler
-

Détail Offre

`/jobs/:id`

Affichage

- Infos client
- Description
- Budget
- Durée
- Deadline
- Skills

Actions

- Postuler
 - Sauvegarder
-

Modal Candidature

Champs :

- Lettre motivation
- Tarif proposé

- Date livraison
- Portfolio

Actions :

- Soumettre
 - Annuler
-

ROLE : ADMIN

Dashboard Offres

`/admin/jobs`

Colonnes :

- ID
- Client
- Catégorie
- Budget
- Statut
- Candidatures
- Date

Actions :

- Voir
 - Suspendre
 - Archiver
-

MODULE 2 : PROJECT (Gestion Projet & Jalons)

ROLE : CLIENT

Créer Projet (après acceptation)

`/projects/create`

Inputs

- Date début
- Date fin
- Exigences
- Livrables
- Section Jalons (dynamique)

Pour chaque jalon :

- Titre
- Description
- Date limite
- Critères d'acceptation
- Livrables attendus

Actions

- Ajouter jalon
 - Réordonner (drag & drop)
 - Créer projet
-

Dashboard Projet

`/projects/:id/dashboard`

Affichage

- Progression globale %
- Timeline horizontale
- Jalons avec statut
- Compteur jalons en attente

Actions

- Voir détail jalon
 - Suspendre projet
 - Annuler projet
-

Détail Jalon

`/projects/:id/milestones/:id`

Affichage

- Description
- Critères
- Livrables soumis
- Historique

Actions (si submitted)

- Approuver
 - Demander révisions
-

ROLE : FREELANCE

Mes Projets

`/my-projects`

Affichage :

- Nom projet
 - Client
 - Progression
 - Statut
 - Deadline
-

Dashboard Projet Freelance

`/my-projects/:id`

Affichage :

- Liste jalons
 - Statut coloré
 - Jalon actif mis en évidence
 - Jours restants
-

Détail Jalon Freelance

`/my-projects/:id/milestones/:id`

Affichage

- Description
- Critères validation
- Deadline

Section Soumission

- Upload fichiers
- Commentaire
- Checkbox validation critères

Action

- Soumettre jalon
-

ROLE : ADMIN

Dashboard Projets

`/admin/projects`

Colonnes :

- Client

- Freelance
- Progression
- Statut
- Jalons validés

Actions :

- Voir
 - Médiation
-

Médiation Jalon

`/admin/projects/:id/milestones/:id/mediate`

Affichage :

- Critères
- Livrables
- Historique échanges

Actions :

- Forcer approbation
- Demander révision
- Annuler jalon

MODULE 3 : RECOMMANDATION MANUELLE FREELANCE

Interface CLIENT

1.1 Rechercher un Freelance

Page

`/freelancers`

Objectif

Permettre au client de chercher et recommander un freelance manuellement.

Navigation

Menu → Freelances → Rechercher

Inputs (Filtres)

- Recherche par mot-clé (nom / skill)
- Compétences (multi-select)
- Niveau d'expérience (beginner / intermediate / expert)
- Tarif minimum / maximum
- Localisation
- Disponible maintenant (checkbox)
- Note minimum

Résultat (Cartes Freelance)

Chaque carte contient :

- Photo
 - Nom
 - Titre professionnel
 - Note ★
 - Nombre de projets complétés
 - Tarif moyen
 - Liste skills (tags)
 - Boutons :
 - Voir Profil
 - Recommander pour une offre
-

1.2 Modal : Recommander un Freelance

Objectif

Associer un freelance à une offre existante.

Inputs

- Sélection de l'offre (dropdown → offres du client)
- Message personnalisé (textarea)
- Budget proposé (optionnel)
- Date limite de réponse (optionnel)

Actions

- Annuler
 - Envoyer recommandation
-

1.3 Page : Mes Recommandations

`/my-recommendations`

Colonnes

- Offre
- Freelance
- Date envoi
- Statut
- Deadline
- Actions

Statuts

- pending
- accepted
- declined
- expired

Actions possibles

- Voir détails

- Annuler (si pending)
 - Relancer
-

1.4 Détail Recommandation

`/my-recommendations/:id`

Affiche :

- Infos Offre
- Infos Freelance
- Message envoyé
- Statut
- Historique

Actions :

- Annuler invitation
 - Envoyer message
-

Interface FREELANCE

2.1 Page : Mes Invitations

`/my-invitations`

Liste

Pour chaque invitation :

- Nom client
- Titre offre
- Budget
- Date réception
- Message
- Statut

Actions

- Accepter et Postuler
 - Refuser
 - Voir détails
-

2.2 Détail Invitation

`/my-invitations/:id`

Affiche :

- Offre complète
- Message client
- Budget proposé
- Date limite réponse

Actions :

- Accepter
 - Refuser
-

Si accepté :

Redirection vers formulaire Application (déjà défini dans ta structure)
tables_joboffer_application_int...

Interface ADMIN

3.1 Dashboard Recommendations

`/admin/recommendations`

Tableau

- ID
- Client
- Freelance

- Offre
- Statut
- Date
- Deadline
- Actions

Filtres :

- Statut
 - Date
 - Client
 - Freelance
-

3.2 Statistiques

`/admin/analytics/recommendations`

KPIs :

- Nombre total invitations envoyées
- Taux acceptation
- Taux refus
- Taux conversion invitation → projet

Graphiques :

- Évolution mensuelle invitations
- Top clients qui recommandent
- Top freelances invités

ARCHITECTURE GLOBALE SANS EMAIL

Système Central de Notifications Temps Réel

Technologie

- WebSocket (Spring Boot + STOMP)

ou

- Socket.io

Canaux

- `/topic/user/{userId}`
- `/topic/job/{jobId}`
- `/topic/project/{projectId}`

Table Notification

- id
 - userId
 - type (JOB, APPLICATION, PROJECT, INVITATION, SYSTEM)
 - title
 - message
 - isRead
 - createdAt
 - referenceld
 - referenceType
-

MODULE JOB OFFER – VERSION AVANCÉE SANS EMAIL

Nouvelle Candidature (Temps Réel)

Quand un freelance postule :

WebSocket push au client

Badge rouge sur icône notification

Compteur candidatures mis à jour live

Statistiques Dynamiques Live

Dashboard client :

- Nombre candidatures live
- Vues en temps réel
- Taux conversion
- Temps moyen réponse

Update via WebSocket.

Recherche Live Instantanée

- Recherche AJAX
 - Suggestions dynamiques
 - Résultats sans reload page
-

Badge Smart Offer

- Offre active
- Offre expirée
- Offre populaire (beaucoup candidatures)

Calcul automatique côté backend.

MODULE PROJECT – VERSION AVANCÉE SANS EMAIL

Progression Live

Quand jalon est approuvé :

Update % progression instantanément

Notification freelance

Animation barre progression

Alertes Deadline In-App

Scheduler backend :

- J-3 → notification
- J-1 → notification urgente
- Retard → badge rouge

Tout visible dans dashboard.

Versioning Livrables

Chaque soumission :

- Version 1
- Version 2
- Historique visible

Pas besoin mail.

Discussion Temps Réel par Projet

Chat intégré :

- WebSocket
 - Message instantané
 - Vu / non vu
 - Upload fichiers
-

MODULE RECOMMANDATION – SANS EMAIL

Invitation Instantanée

Quand client recommande :

Notification WebSocket au freelance

Badge rouge "Nouvelle invitation"

Expiration Automatique

Cron job :

- Si deadline dépassée
 - Status = expired
 - Notification in-app
-

Stats Invitations

Dashboard :

- Taux acceptation
- Temps réponse moyen
- Invitations en attente

Live update.

MODULE GLOBAL AVANCÉ

Score Réputation Dynamique

Calcul basé sur :

- % projets terminés
- % jalons validés 1er essai
- Respect deadlines
- Temps réponse

Update automatique.

Dashboard Admin Temps Réel

- Offres actives
- Projets en retard
- Invitations en attente
- Top freelances actifs

Tout en live.

Audit Log

Chaque action stockée :

- Création offre
- Acceptation projet
- Rejet jalon
- Annulation

Historique complet.

Architecture Technique Propre

Frontend:

- Angular
- WebSocket client
- Store central (Redux / NgRx)

Backend:

- Spring Boot
- WebSocket config
- Scheduler
- REST APIs

Base:

- MySQL
- Redis (optionnel cache notifications)